

Review of Object Oriented Programming (OOP)

1. Introduction

■ NOTE

1. As observed in the Overview, **Object-oriented programming (OOP)** is an important component of Integrative development
2. This is a review of OOP
3. The programming language in this course is Java
 - The OOP concepts are independent of programming language
 - However, all examples and exercises are in Java

2. Objects

- An object has **properties** and **performs actions**
 1. Properties (or fields): **Data**
 2. Actions (or operations, functions): **Methods**
- An object is a “self-contained” unit (i.e., standalone and independent)
- Objects work together (**interact**) to solve a problem
 1. Objects interact by **calling the methods**
 2. One object can **request another to perform an action**
 3. Example: **Lecturer** can call methods of **Student** (i.e., **Lecturer** object can ask **Student** object to perform actions)



3. Encapsulation and data hiding

- Encapsulation: grouping **members** (i.e., **data and methods**) to form a standalone object
 - **Rule**: Increase cohesion and reduce coupling
- Data hiding restricts which members other objects can and cannot access
 1. This reduces unintended data changes by other objects
 2. E.g., if access to **name** and **student Id** of a Student object are restricted:
 - a. Lecturer object **cannot** make changes
 - b. Lecturer object must **request** Student object (by calling a Student method) to make any changes
- Objects are encapsulated as **classes**
- Data hiding is implemented as **access levels**
- **NOTE:**
 1. We also have **method hiding**
 2. **Abstraction** refers to data and method hiding

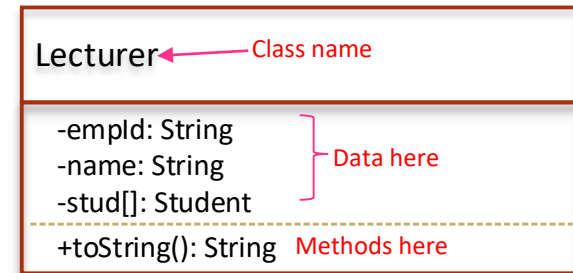
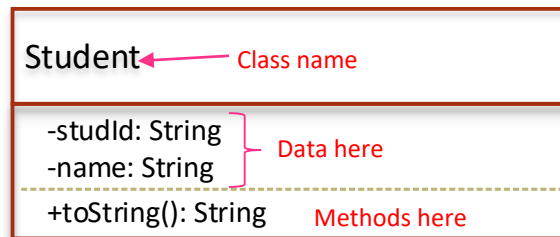
3. Encapsulation and data hiding

■ Benefits

1. Integrity: independent objects reduces unintended modification
2. Code re-use
 - a. Objects can be used as “Abstract Data Types” (ADT)
 - b. E.g., Lecturer can use Student as primitive (without knowing Student implementation)
3. Easy maintenance: changes to one does not affect others
4. Any other?
5. All the above save development time and cost

4. Objects as classes

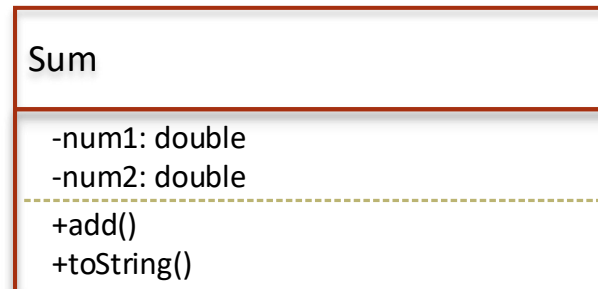
- Objects are represented as **UML classes** and show the **class name**, **members** and **access levels**
- Example: UML class diagrams of Student and Lecturer



- There are different types of classes, e.g., **super**, **sub**, and **abstract classes** (**Next lecture**)
- There are different types of data (e.g., **integer**, **decimal**, **string** and **ADT** (i.e., objects))
- There are different types of methods, e.g., **constructors**, **get** and **set**, **special** methods (e.g. add, draw, etc.)
- The main access levels are: **-**, **#**, **~** and **+** (to be discussed later)

5. Class diagram to Java program

- Example 1: Write a program to add two numbers and display their sum.
- Steps
 1. Identify members (**data** and **methods**) and draw UML class diagram for the object



2. Convert the UML class diagram to Java class (**POJO**)
3. Write a program (**tester**) with **main()** that creates and uses **instances** of the class Sum above

5. Class diagram to Java program

- NOTE: The Java class (**POJO**) for the UML class must contain:
 1. **Constructors**
 2. **Get** and **set** (or accessor and setter) methods
 3. **Special** methods (e.g., add, toString(), etc.)

5. Class diagram to Java program (POJO)

```
package sum;
public class Sum {
    //Declare data
    private double num1;
    private double num2;

    //Define the methods
    public Sum() {
        this.num1 = 0;
        this.num2 = 0;
    }
    public Sum(double num1, double num2) {
        this.num1 = num1;
        this.num2 = num2;
    }
    public double getNum1() {
        return num1;
    }
    public void setNum1(double num1) {
        this.num1 = num1;
    }
    public double getNum2() {
        return num2;
    }
    public void setNum2(double num2) {
        this.num2 = num2;
    }
    public double add(){
        return num1+num2;
    }
    public String toString() {
        return "The sum of " +num1 +" and " +num2+
            " is "+ add();
    }
}
```

5. Class diagram to Java program (Tester)

```
package sum;
public class SumTester {

    public static void main(String[] args) {
        //create instances of the class and use to call its methods
        Sum s1=new Sum();
        System.out.println(s1.toString());
        Sum s2=new Sum(20, 30);
        System.out.println(s2);
    }
}
```

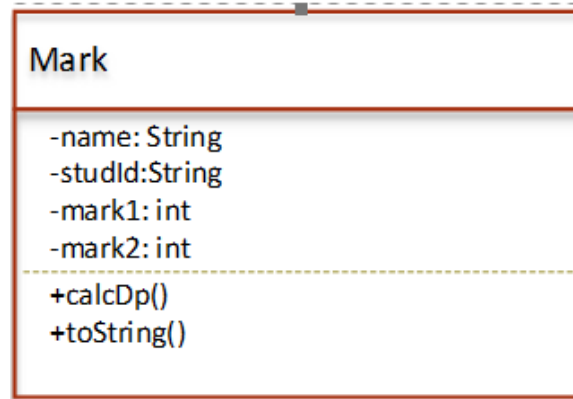
Create objects (i.e.,
instances of class)

Use the object
(print it)

Use the object (to
call its method)

6. Exercise

■ Complete Exercise 3 (**Review exercises**)



- `calcDp()` returns the dp (average mark)
- Convert class **Mark** to a Java program
- Write a “tester” program (**Tester**) to create and display at least two instances of **Mark**.

6. Exercise

- Complete Exercise 4 (**Review exercises**)

7. Summary

We have covered the basic concepts of OOP:

1. Object, encapsulation and classes
2. Benefits of encapsulation and data hiding
3. Objects as UML class diagrams
4. Why separate the object class and tester
5. How to write an object and tester classes in Java
6. How to instantiate and use objects of a class

8. Questions and comments

?

Next.....

- Topic 1: Part 1 (Inheritance)